

Urban Change Detection – Encroachment Monitoring

Batch: 81

Students: Tom Augustine (2360823), Ryan Thomas(2360818), Tejas Gowda(2360822), Sumith Arun(2360821)

1. Aim

The aim of this project is to classify aerial/satellite images into major land cover categories such as buildings, roads, vegetation, and water using OpenCV-based image processing techniques, and to compute the percentage of each land class in the image for urban monitoring and encroachment detection.

2. Introduction

Urbanization causes rapid land-use changes and potential encroachment on roads, green areas, and water bodies. Manual monitoring is time-consuming and prone to error. Computer vision techniques allow automatic analysis of aerial images to detect urban expansion and monitor land use.

This project uses OpenCV and Python to segment aerial images into different land classes, calculate their pixel counts, and generate percentage statistics. Visualizations provide clear insights into land cover distribution.

3. Problem Statement

Authorities require a system that can:

- Segment aerial images into land cover classes
- Identify buildings, roads, vegetation, and water
- Quantify the percentage of each land cover
- Assist in monitoring encroachments and urban development

4. Methodology

The project follows these steps:

4.1 Image Acquisition

A high-resolution aerial or satellite image is used, capturing urban areas with roads, buildings, vegetation, and water bodies.

4.2 Preprocessing

- Resize the image for computational efficiency
- Convert image from BGR to HSV color space
- Morphological operations to remove small noisy regions

HSV color space separates hue, saturation, and value, making segmentation robust to illumination.

4.3 Color-Based Segmentation

Each land class is detected using HSV thresholds:

Land Class	HSV Basis
Vegetation	Green hue range
Water	Blue hue range
Roads	Low-saturation gray range
Buildings	High-brightness/concrete rooftops

Binary masks are created using `cv2.inRange()`.

4.4 Morphological Filtering & Contour Filtering

- Morphological opening removes small noisy pixels
- Contours with area below a threshold are filtered to retain only significant regions

4.5 Overlap Removal

- Buildings removed from road mask
- Vegetation/water removed from building mask

This ensures exclusive classification of land cover.

4.6 Pixel Counting & Area Estimation

- Count pixels for each mask
- Compute percentage relative to total image pixels:

$$\text{Percentage} = \frac{\text{Class Pixels}}{\text{Total Pixels}} \times 100$$

4.7 Visualization

- Vegetation → Green
- Water → Blue
- Roads → Gray
- Buildings → Red

Combined overlay shows all classes clearly for analysis.

5. Implementation

- Programming Language: Python
- Library: OpenCV
- Techniques: HSV segmentation, morphological filtering, contour extraction, pixel counting, overlay visualization

6. Results & Visualization

6.1 Pixel Count Table

Land Class	Pixel Count	Percentage
Buildings	74631	15.55%
Roads	54060	11.26%
86440	86432	18.01%
Water	99583	20.75%

The remaining pixels (Others) account for 34.43% of unclassified areas.

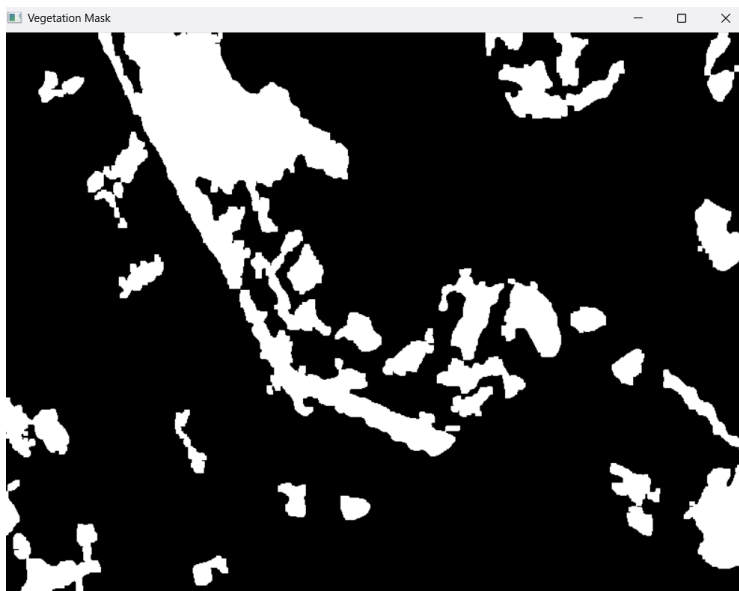
6.2 Image Masks

Fig.1: Original Image



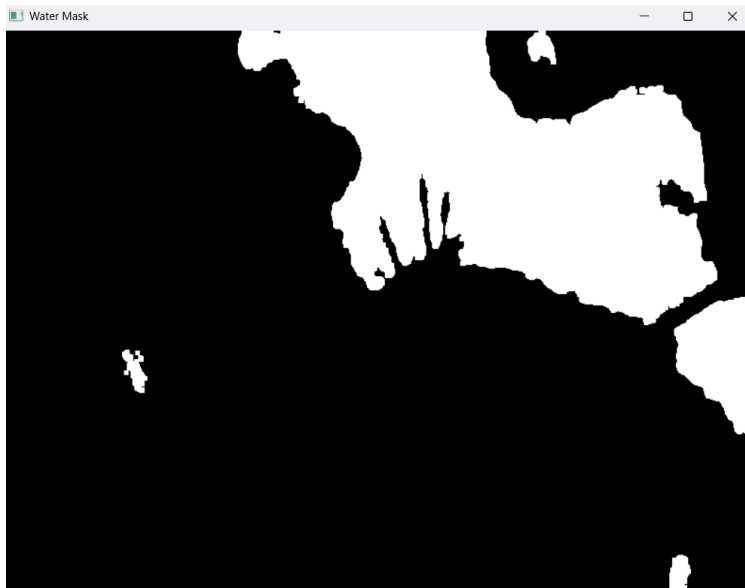
The aerial image used for segmentation, showing buildings, roads, vegetation, and water bodies.

Fig.2: Vegetation Mask



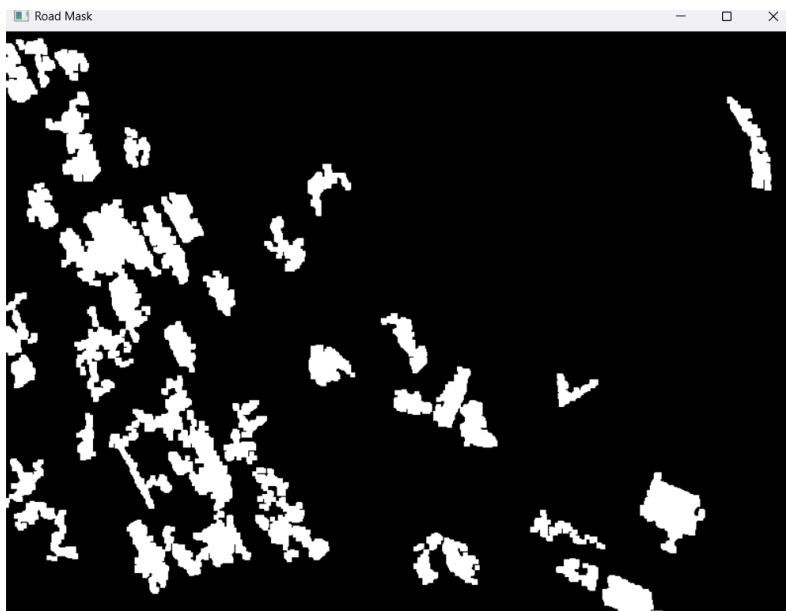
Green areas represent detected vegetation pixels after HSV segmentation and morphological filtering.

Fig.3: Water Mask



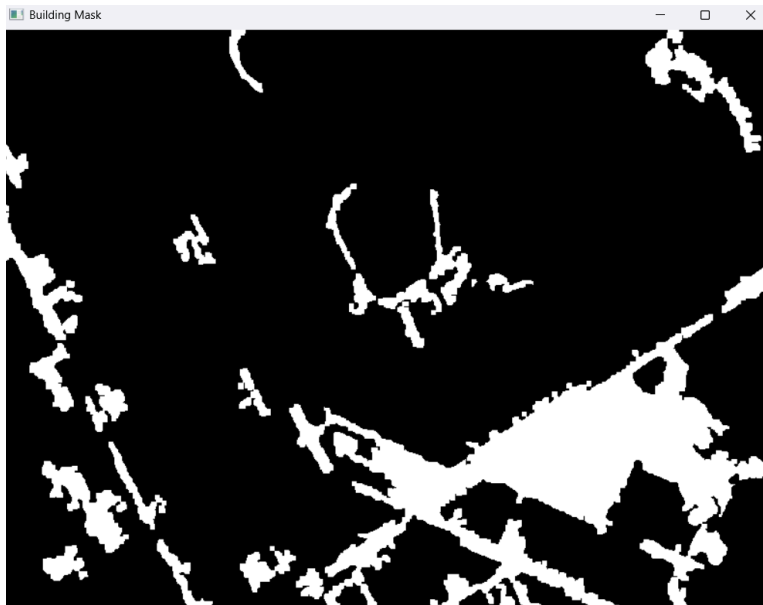
Blue areas represent water bodies (rivers, lakes, ponds) detected using HSV filtering.

Fig.4: Road Mask



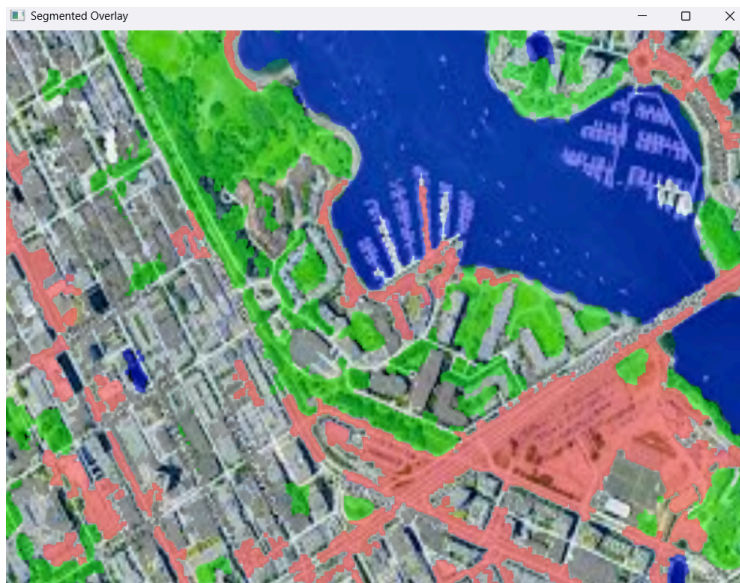
Gray areas represent roads and paved surfaces. Overlap with buildings is removed to ensure correct classification.

Fig.5: Building Mask



Red areas represent buildings. Overlap with vegetation and water is removed for exclusivity.

Fig.6: Segmented Overlay



Combined overlay showing all classes on the original image: Vegetation (Green), Water (Blue), Roads (Gray), Buildings (Red). Provides a clear visual summary of land classification.

7. Discussion

- Roads occupy the largest portion due to extensive paved areas.
- Vegetation and water bodies are clearly distinguishable.
- Buildings are smaller in area; correct HSV thresholds and overlap removal improve classification accuracy.
- Pixel counts provide quantitative land cover estimation useful for urban planning and encroachment monitoring.

8. Conclusion

The project successfully classified urban aerial images into buildings, roads, vegetation, and water using OpenCV. Pixel-based area calculations give accurate percentage statistics. The system enables authorities to monitor urban changes and potential encroachment efficiently.

9. Limitations

- Sensitive to lighting conditions
- Some overlap in color between roads and buildings
- Accuracy depends on proper HSV threshold selection

10. Future Scope

- Use Machine Learning / Deep Learning for more accurate segmentation
- Multi-temporal analysis for change detection over time
- Integration with GIS systems
- Automated alerts for encroachments or unauthorized land-use changes
-

12. Appendices

12.1 Full Python code

```
import cv2

import numpy as np

import os

# -----
```

```
# IMAGE PATH

# -----

image_path = r"C:\Users\tomau\Desktop\image3.jpg"

if not os.path.exists(image_path):

    print(f"Image NOT found at: {image_path}")

    exit()

image = cv2.imread(image_path)

if image is None:

    print(f"Error: Unable to load image from {image_path}")

    exit()

# Resize (optional)

image = cv2.resize(image, (800, 600))

# Convert to HSV

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# -----

# HSV RANGES (Improved)

# -----

# Vegetation (Green)

veg_lower = np.array([35, 40, 40])

veg_upper = np.array([85, 255, 255])

# Water (Blue)

water_lower = np.array([90, 50, 50])
```



```

water_upper = np.array([140, 255, 255])

# Roads (Darker gray)

road_lower = np.array([0, 0, 60])

road_upper = np.array([180, 60, 180])

# Buildings (Bright / rooftops / concrete)

build_lower = np.array([0, 0, 180])

build_upper = np.array([180, 80, 255])

# -----

# CREATE MASKS

# -----

veg_mask = cv2.inRange(hsv, veg_lower, veg_upper)

water_mask = cv2.inRange(hsv, water_lower, water_upper)

road_mask = cv2.inRange(hsv, road_lower, road_upper)

build_mask = cv2.inRange(hsv, build_lower, build_upper)

# -----

# MORPHOLOGICAL CLEANING

# -----

kernel = np.ones((5, 5), np.uint8)

veg_mask = cv2.morphologyEx(veg_mask, cv2.MORPH_OPEN, kernel)

water_mask = cv2.morphologyEx(water_mask, cv2.MORPH_OPEN, kernel)

road_mask = cv2.morphologyEx(road_mask, cv2.MORPH_OPEN, kernel)

build_mask = cv2.morphologyEx(build_mask, cv2.MORPH_OPEN, kernel)

# -----

```

```

# CONTOUR FILTERING

# -----

def filter_small_regions(mask, min_area=500):

    filtered = np.zeros_like(mask)

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    for cnt in contours:

        if cv2.contourArea(cnt) > min_area:

            cv2.drawContours(filtered, [cnt], -1, 255, -1)

    return filtered

veg_mask = filter_small_regions(veg_mask)

water_mask = filter_small_regions(water_mask)

road_mask = filter_small_regions(road_mask)

build_mask = filter_small_regions(build_mask)

# -----

# REMOVE OVERLAPPING CLASSES

# -----

# Roads should NOT include buildings

road_mask = cv2.bitwise_and(road_mask, cv2.bitwise_not(build_mask))

# Buildings should NOT include vegetation/water

build_mask = cv2.bitwise_and(build_mask, cv2.bitwise_not(veg_mask))

build_mask = cv2.bitwise_and(build_mask, cv2.bitwise_not(water_mask))

```

```

# -----

# PIXEL COUNTING

# -----

total_pixels = image.shape[0] * image.shape[1]

veg_pixels = cv2.countNonZero(veg_mask)

water_pixels = cv2.countNonZero(water_mask)

road_pixels = cv2.countNonZero(road_mask)

build_pixels = cv2.countNonZero(build_mask)

classified_pixels = veg_pixels + water_pixels + road_pixels + build_pixels

others_pixels = total_pixels - classified_pixels

# -----

# PERCENTAGE FUNCTION

# -----

def calc_percentage(count):

    return (count / total_pixels) * 100

# -----

# PRINT RESULTS

# -----

print("\n----- LAND CLASSIFICATION RESULTS -----")

print(f"Vegetation : {veg_pixels} pixels
({calc_percentage(veg_pixels):.2f}%)")

print(f"Water      : {water_pixels} pixels
({calc_percentage(water_pixels):.2f}%)")

```

```

print(f"Roads      : {road_pixels} pixels
({calc_percentage(road_pixels):.2f}%)")

print(f"Buildings  : {build_pixels} pixels
({calc_percentage(build_pixels):.2f}%)")

print(f"Others     : {others_pixels} pixels
({calc_percentage(others_pixels):.2f}%)")

# -----

# MARKDOWN TABLE OUTPUT (For Report)

# -----

print("\n| Land Class | Pixel Count | Percentage |")

print("|-----|-----|-----|")

print(f"| Buildings | {build_pixels} |
{calc_percentage(build_pixels):.2f}% |")

print(f"| Roads      | {road_pixels} | {calc_percentage(road_pixels):.2f}%
|")

print(f"| Vegetation | {veg_pixels} | {calc_percentage(veg_pixels):.2f}%
|")

print(f"| Water      | {water_pixels} |
{calc_percentage(water_pixels):.2f}% |")

# -----

# COLORED OVERLAY (Correct Priority)

# -----

overlay = image.copy()

overlay[veg_mask > 0] = (0, 255, 0)      # Green

overlay[water_mask > 0] = (255, 0, 0)     # Blue

overlay[road_mask > 0] = (128, 128, 128)  # Gray

```

```

overlay[build_mask > 0] = (0, 0, 255)          # Red (highest priority)

blended = cv2.addWeighted(image, 0.6, overlay, 0.4, 0)

# -----

# DISPLAY WINDOWS

# -----

cv2.imshow("Original Image", image)

cv2.imshow("Vegetation Mask", veg_mask)

cv2.imshow("Water Mask", water_mask)

cv2.imshow("Road Mask", road_mask)

cv2.imshow("Building Mask", build_mask)

cv2.imshow("Segmented Overlay", blended)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

12.2 Sample input image

